Integration with Zomato Logistics APIs - External Documentation

Using the Zomato Delivery API, developers can easily integrate Zomato delivery platform into their applications as B2B partner. The API is designed to allow application developers to book a delivery, follow updates on delivery till completion etc.

Merchant Registration

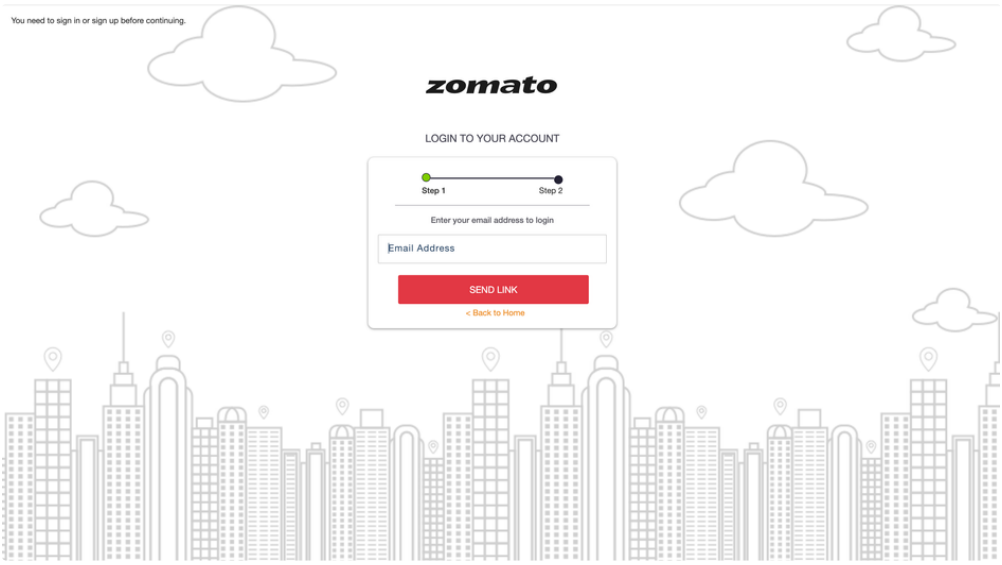
To use Zomato delivery API, First step is to register the Merchant/ B2B partner in our system. Zomato team can onboard new merchant in few minutes. We will need only below information :

- **Merchant Info** : E-mail address, Phone no
- **Address Details** : Address of Merchant.
- **Category**: To which category the orders placed will belong to.

Once Onboarding is complete, Merchant gets confirmation mail to the registered email id, and can login in our Merchant Dashboard. Follow the below steps to login to merchant dashboard :-

1. Proceed with “Log in With Email”.

2. Please enter the registered email id and click on “SEND LINK”.



3. Please follow the steps provided in the received mail.

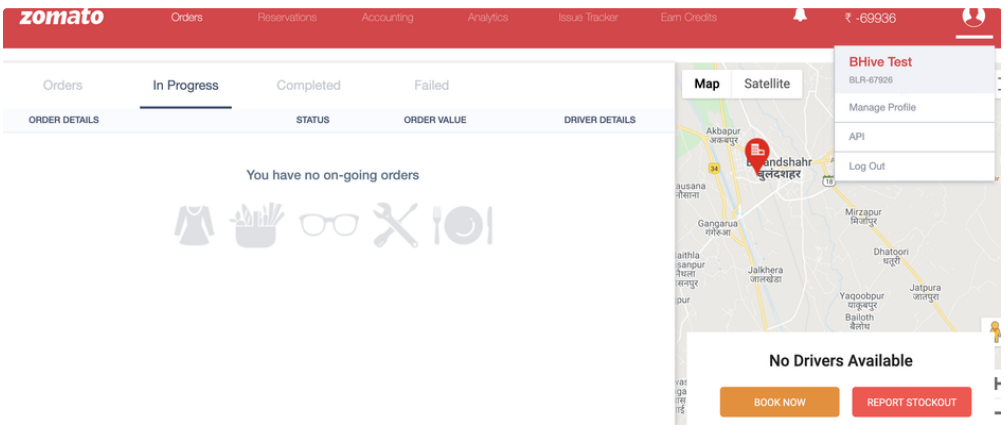
Our APIs are REST-based. This means:

- It make use of standard HTTP verbs like GET, POST, DELETE.
- The API uses standard HTTP error responses to describe errors.

Getting Started

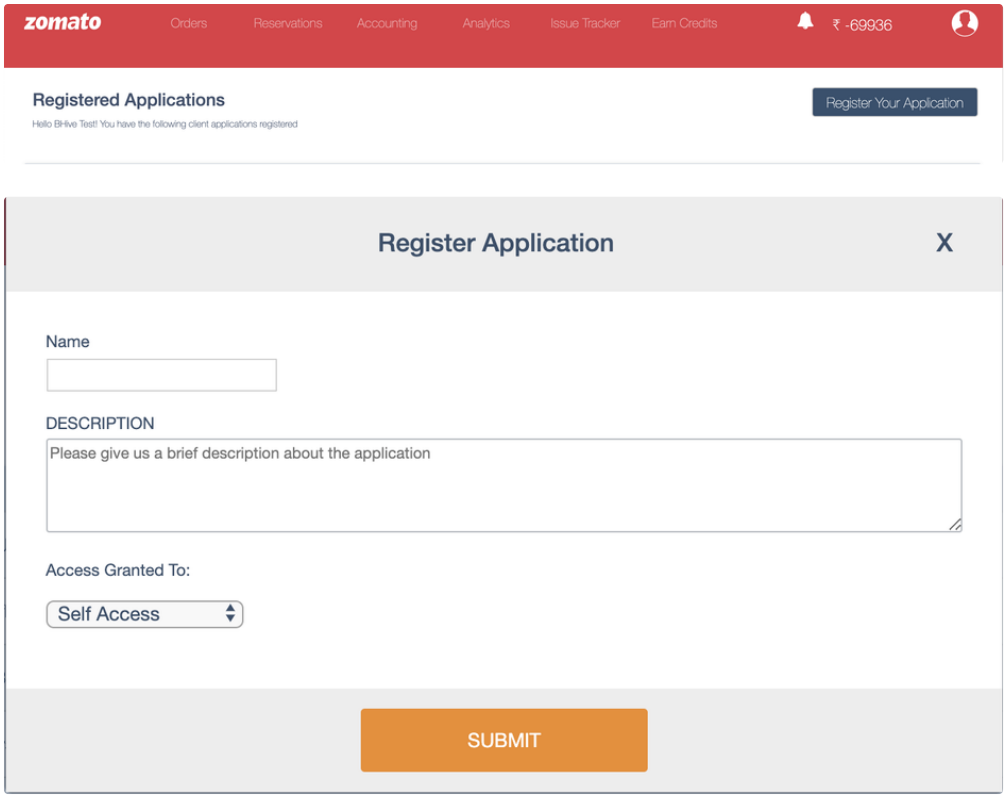
To get started using the Zomato Delivery API, B2B partners needs to request the **Client Id and Secret**. Step are as below :-

- Step #1 : To create new Client Id and Secret, Please login to Merchant dashboard and click on API menu (as shown in below screenshot). Link : Zomato /merchant#!/api



- Step #2 After Clicking on API button, Click on Register Your Application and fill the required info.

```
Name: {{Merchant Name}}
Description: API integration
Access Granted To : Self Access
```



An email containing **Client Id and Secret** will be sent to the registered email id upon successful registration.

Generate the **API Token**

The Zomato Delivery APIs requires authentication token generated through OAuth 2.0. You need to call the following API to generate the access token.

The access_token has an expiry as provided in the response , you can use the same API after expiry to refresh the token.

Generate API Token

```
HTTP Method: GET
Endpoint: <host_url>/oauth/token?grant_type=client_credentials&
client_id=<client_id>&client_secret=<client_secret>
Request : Use client_id and client_secret generated for your account.
```

Generate API Token Response

```
{
 "access_token": " Djuskljfsldjflaksja",
 "token_type": "bearer",
 "expires_at": "2020-07-04T09:56:18.00+05:30"
}
```

Once token is generated, developers can start actual order api calls.

Zomato Delivery APIs

To see if order delivery is possible, call our **Serviceability API**. This endpoint provide information about a potential delivery. After the serviceability API returns TRUE as response you call the **"Create Order" API.** While a delivery is in progress, you can track its status in real-time by polling the Track API, or with webhooks.

Follow a step-by-step guide to place order in Zomato logistics system:

1. Check serviceability, if Driver is available for delivery / the location is serviceable
2. Place order using create API
3. Edit Order API to change the payment type / order value
4. Cancel API for cancellation
5. Order status callbacks
6. Order Track API

Pre-Requisite :

- Valid API token for api calls.

Host URL: `https://api.runnr.in`

Order Serviceability API

A. API Contract

Order Serviceability API Header

```
HTTP Method: POST
Endpoint : <host_url>/v1/orders/serviceability
Header : {
  Content-Type : application/json
  Authorization : {{access_token}}
}
```

Order Serviceability Realtime API Request

```
{
  "pickup":{
    "user":{
      "full_address":{
        "address":"28, 4th 'B' Cross",
        "geo":{
          "latitude":"28.4882497",
          "longitude":"77.0865653"
        }
      }
    }
  },
  "drop":{
    "user":{
      "full_address":{
        "address":"28, 4th 'B' Cross",
        "geo":{
          "latitude":"28.4882497",
          "longitude":"77.0865653"
        }
      }
```

```
    }
  },
  "order_details":{
    "amount_to_be_collected": 100,
    "order_id": "234567897"
  }
}
```

Order Serviceability Realtime API Response

```
{
    "status": {
        "code": 200
    },
    "serviceability": {
        "B2B": {
            "serviceable": true,
            "average_assignment_minutes": 3.5,
            "rider_pickup_time": 3.4
        }
    },
    "estimated_service_fee":{
        "total_amount": 100,
        "breakdown":{
          "distance_charge": 79.5,
          "surge_charge": 20.5
        }
    }
}
```

pickup is mandatory.

To fetch estimated service fee details(estimated_service_fee), drop & order_details are required.

Order Create API

A. API Contract

Order Create API Header

```
HTTP Method: POST
Endpoint : <host_url>/v1/orders/ship
Header : {
  Content-Type : application/json
  Authorization : {{access_token}}
}
```

Order Create API Request

```
{
   "pickup":{
      "user":{
         "name":"Thali Inbox",
         "phone_no":"8368137508",
         "full_address":{
            "address":"Zomato Office farmhouse, Gurgaon",
```

```
            "geo":{
               "latitude":"28.4882497",
               "longitude":"77.0865653"
            }
         },
         "type":"merchant"
      }
   },
   "drop":{
      "user":{
         "name":"Test User",
         "phone_no":"8335881012",
         "full_address":{
            "address":"First Floor, KP Block, Gurgaon",
            "geo":{
               "latitude":"28.4892587",
               "longitude":"77.0842583"
            }
         },
         "type":"customer"
      }
   },
   "order_details":{
      "order_value":"150",
      "order_type":{
         "name":"Prepaid" // Prepaid or CashOnDelivery
      },
      "category":{
         "name": "FOOD"
      },
      "order_id":"Test84fnf4039",
      "amount_to_be_collected":"0", // set to 0 if order is Prepaid
      "pickup_otp": "3746",
      "drop_otp": "8734",
      "return_otp": "4321",
      "preparation_time": 15, //In minutes
      "drop_user_instruction": {"text": "Handle with care"},
      "order_items":[
         {
            "item":{
               "name":"Chai"
            },
            "weight":300,
            "quantity":3,
            "price": 50
         },
         {
            "item":{
               "name":"Besan"
            },
            "weight":1000, //In grams
            "quantity":1,
            "price":100
         }
      ]
   },
   "created_at":"2020-03-06 17:09",
```

```
    "callback_url": "http://requestupdate.in/w57o5aw5"
}
```

Order Create API Response

- Successful Response

```
{
    "status": {
        "code": 200,
        "message": "Order Created SuccessFully!! It Will Be Process Shortly"
    },
    "external_order_id": "Test84fnf4039",
    "order_id": "orhshebfakdhfh",
    "internal_order_id": "orhshebfakdhfh"
}
```

- No Driver is available for delivery

```
{
    "status": {
        "code": 706,
        "message": "No driver is available."
    },
    "handshake_type": "normal",
    "external_order_id": "P90917869"
}
```

- Auth Token is invalid

```
HTTP Token: Access denied.
```

B. Order Create Request Fields

Field Name	Type	Mandatory	Usage
pickup.user	Object	Yes	Contains the detail of pickup user, all mentioned fields are mandatory
drop.user	Object	Yes	Contains the detail of drop user, all mentioned fields are mandatory
order_details	Object	Yes	Order item information and payment type
callback_url	String	No	If provided, will be used for order status callback
order_details.promised_sla	integer	No	After LPTO(logistics partner timeout), system will not delivery this order. `LPTO` = max(edt-rdt, 30) `edt` => estimated delivery time is also called `promised_sla` `rdt` => rider delivery time is called as rider delivery time

order_details.category.name	String	No	If a merchant is mapped to multiple categories, category name should be mentioned here.
order_details.order_items.weight	Float	No	(In grams) It should be the total weight of the item. Eg: one Chai weighs 100gms but 3 will weigh 300gms.

API Description

Request Fields

```
    "user":{
      "name":"Mandatory, Name of Pickup location - shown in Rider app",
      "phone_no":"Mandatory, Mobile number of Pickup location.
      Rider may call to this number if required",
      "full_address":{
        "address":"Mandatory, Address of pickup location - shown in Rider app",
        "city":{
          "name":"Mandatory, City of pickup location - shown in Rider app"
        },
        "geo": "Mandatory, used for showing path in Rider App."
      },
      "type": "Mandatory, It must be 'merchant'"
    }

"order_details":{
      "order_value":"Mandatory, Total cost of order",
      "order_type":{
         "name":"CashOnDelivery/Prepaid"
      },
      "category": {
         "name": "category name should be mentioned here if merchant has
         multiple categories mapped"
      },
      "order_id":"Order Id of third party",
      "amount_to_be_collected":"Should be amount to be collected from customer
      for CashOnDelivery. Must be 0 for Prepaid order.",
      "amount_to_be_paid":"Must be 0",
      "pickup_otp": "OTP to be used at merchant for order pickup handshake",
      "drop_otp": "OTP to be used for drop order handshake with customer.",
      "return_otp": "OTP to be used for return order handshake.",
      "order_items":[
         {
            "item":{
               "name":"Chai"
            },
            "weight":300,
            "quantity":3,
            "price": 50
         },
         {
            "item":{
               "name":"Besan"
            },
            "weight":1000
            "quantity":1,
            "price":100
         }
      ]
```

```
  },
```

C. Order Create Response Fields

Field Name	Possible Values	Usage
status	200 / 706	Constrains status code and message
external_order_id		Same as in Request API

Order Edit API

This API has to be used if any prepaid orders has to be converted to COD or vice versa.
Use this API only for ongoing & not delivered orders.

A. API Contract

Order Edit API Header

```
HTTP Method: POST
Endpoint: <host_url>/v1/orders/{internal_order_id}/edit
Header : {
  Content-Type : application/json
  Authorization : {{access_token}}
}
```

Order Edit API Request

```
For Prepaid to COD / COD to COD(amount update)
{
  "order_details": {
    "amount_to_be_collected": "500",
    "order_value": "500",
    "amount_to_be_paid": "0",
    "order_type": {
      "name": "CashOnDelivery"
    }
  }
}

For COD to Prepaid
{
  "order_details": {
    "amount_to_be_collected": "0",
    "order_value": "500",
    "amount_to_be_paid": "0",
    "order_type": {
      "name": "Prepaid"
    }
  }
}
```

Order Edit API Response

- Order is cancelled successfully.

```
{
    "status": {
        "code": 200
    }
}
```

- Order is not editable.

```
{
    "status": {
        "code": 500
    },
    "errors": "Order is not editable"
}
```

- Auth Token is invalid

```
HTTP Token: Access denied.
```

Order Cancellation API

We support following cancellation reasons :

Cancellation Reason ID	Cancellation Reason
4	"Customer cancelled"
5	"Delivery driver refused to come"
6	"Delivery driver did not came for long time"
9	"Other (Open)"
48	"KPT is more than 20 Mins"
51	"Merchant outlet is closed"
52	"Merchant requested"
53	"No other driver to reassign"
124	"Item out of stock"
125	"Outlet closed"
126	"Customer cancellation"
127	"Customer reject due to delay"
128	"Unsafe area"
129	"Mx - No answer"
130	"Wrong user address"

131	"FE - Accident / Rain / Strike / vehicle issues"
132	"Customer non-responsive"
139	"Heavy Rain"
140	"Strike"
141	"Customer location in unsafe area"
142	"Vehicle Issue"
172	"Poor packaging or spillage"
173	"Valet is sick or unwell"
174	"Wrong order or missing items"
175	"Valet device or app issue"
176	"Customer refuse to pay"
177	"Others"
184	"Customer doesn't need the order"
185	"Few items are unavailable in the order"
186	"Customer selected incorrect address"
187	"Payment issue at doorstep"
188	"Customer denied order at doorstep"
189	"Customer not responding"
191	"Rider SOS"
196	"Wrong drop address"
197	"Wrong pick up address"
198	"Driver not assigned yet"
199	"Requested by mistake"
200	"Pick up time is too high"
202	"Pick up delayed"
203	"Driver not moving closer"
204	"Driver requested cancellation"
205	"Unable to contact driver"

A. API Contract

Order Cancellation API Header

```
HTTP Method: POST
Endpoint : <host_url>/v1/orders/{internal_order_id}/cancel
Header : {
```

```
  Content-Type : application/json
  Authorization : {{access_token}}
}
```

Order Cancellation API Request

```
{
  "cancellation_reason_id": 126,
  "notes": "Cancellation notes"
}
```

Order Cancellation API Response

- Order is cancelled successfully.

```
{
    "status": {
        "code": 200
    },
    "external_order_id": "P90917869"
}
```

- Order is already cancelled.

```
{
    "status": {
        "code": 301,
        "message": "Sorry, you cannot cancel the given order.
        Please call us if you need more information."
    },
    "external_order_id": "P90917869"
}
```

- Auth Token is invalid

```
HTTP Token: Access denied.
```

B. Order Cancellation API Request

```
<host_url>/v1/orders/{id}/cancel
```

Field Name	Type	Mandatory	Usage
id	Number	Yes	Zomato Internal Order id
cancellation_reason_id	Number	Yes	Cancellation Reason from the list
notes	String	No	Cancellation reasons

C. Order Cancellation API Response

Field Name	Type	Possible Values	Usage
status	Object	200 / 301	200 for success and 301 if already cancelled.

Order Track Callbacks

Order status lifecycle <> callback

```
TRIP_ASSIGNED - Rider is assigned to the order
DRIVER_ASSIGNED - Rider accepted the order
REACHED_PICKUP - Rider reached pickup
SHIPPED - Rider picked the order.
DRIVER_REACHED_DROP - Rider reached drop
COMPLETE - Order successfully delivered
RETURN_START - Order return start
RETURN - Order return successful
CANCELLED - Order Cancelled
RETURN_CANCELLED - Order return cancelled
DRIVER_ASSIGN_FAILED - when order can't be fulfilled by zomato
RAIN - if it rains during the order.

Successful order:
TRIP_ASSIGNED
DRIVER_ASSIGNED
REACHED_PICKUP
SHIPPED
DRIVER_REACHED_DROP
COMPLETE

Cancel order after pickup -> RTO flow:
TRIP_ASSIGNED
DRIVER_ASSIGNED
REACHED_PICKUP
SHIPPED
RETURN_START
DRIVER_ASSIGNED
REACHED_PICKUP
RETURN

Cancel order after reached drop -> RTO flow:
TRIP_ASSIGNED
DRIVER_ASSIGNED
REACHED_PICKUP
SHIPPED
DRIVER_REACHED_DROP
RETURN_START
DRIVER_ASSIGNED
REACHED_PICKUP
RETURN

Return cancelled:
TRIP_ASSIGNED
DRIVER_ASSIGNED
REACHED_PICKUP
SHIPPED
RETURN_START
DRIVER_ASSIGNED
RETURN_CANCELLED

Cancel order before assignment:
CANCELLED
```

```
Cancel order after assignment:
DRIVER_ASSIGNED
CANCELLED

Cancel order before pickup:
DRIVER_ASSIGNED
REACHED_PICKUP
CANCELLED
```

A. API Contract

Order Track Callbacks API Header

```
HTTP Method: POST
Endpoint : [Callback endpoint to be Provide by merchant/B2B partner]
Timeout : 5 Sec
Header : {
  Content-Type : application/json
  checksum : MD5 hash of [order.internal_order_id + API token]
}
```

Order Track Callbacks API Request

```
{
  driver_name: driver_name,
  driver_number: driver_phone_no,
  timestamp: current_time,
  order_id: order_internal_id,
  reason: reason of return # in case of RETURN
  is_return: true/false
  status: order state (DRIVER_ASSIGNED/REACHED_PICKUP/SHIPPED/DRIVER_REACHED_DROP
  /COMPLETE/CANCELLED/RETURN_START/RETURN/DRIVER_ASSIGN_FAILED/RAIN)
}
```

Order Track Callbacks API Response

```
{}
# We call caller callback API in async.
# We do not use API response anywhere and retry 4 times in case of failure.
```

B. Order Track Callback API Request

Field Name	Type	Mandatory	Usage
driver_id	String		Driver id of Driver in Zomato system
driver_internal_id	Number		Internal Driver id of Driver in Zomato system
driver_name	String	Yes	Driver's Name
driver_number	Number	Yes	Driver Mobile Number
timestamp	Time		Time of API call

order_id	Number		Zomato Internal order id
status	String	Yes	State of Order
reason	String		Reason, In case of cancellation

Order Track Callbacks - Examples

- `DRIVER_ASSIGNED`

```
{
  "order_id": "a4cbfdjhb39faae0ce",
  "status": "DRIVER_ASSIGNED",
  "driver_name": "Avdhesh",
  "driver_number": "956XXX1060",
  "is_return": false,
  "timestamp": 1592389626,
  "external_order_id": "616843374"
}
```

- `REACHED_PICKUP`

```
{
  "order_id": "a4cbfdjhb39faae0ce",
  "status": "REACHED_PICKUP",
  "driver_name": "Avdhesh",
  "driver_number": "956XXX1060",
  "is_return": false,
  "timestamp": 1592389626,
  "external_order_id": "616843374"
}
```

- `SHIPPED`

```
{
  "order_id": "a4cbfdjhb39faae0ce",
  "status": "SHIPPED",
  "driver_name": "Avdhesh",
  "driver_number": "956XXX1060",
  "timestamp": 1592390841,
  "is_return": false,
  "external_order_id": "616843374"
}
```

- `DRIVER_REACHED_DROP`

```
{
  "order_id": "a4cbfdjhb39faae0ce",
  "status": "DRIVER_REACHED_DROP",
  "driver_name": "Avdhesh",
  "driver_number": "956XXX1060",
  "timestamp": 1592391814,
  "is_return": false,
  "external_order_id": "616843374"
}
```

- `COMPLETE`

```
{
  "order_id": "a4cbfdjhb39faae0ce",
  "status": "COMPLETE",
  "driver_name": "Avdhesh",
  "driver_number": "956XXX1060",
  "timestamp": 1592391848,
  "is_return": false,
  "rider_latitude": "26.9142333",
  "rider_longitude": "80.9481367",
  "external_order_id": "616843374"
}
```

Order Track API

Rider's current location(latitude, longitude) is published with certain frequency from rider's device which will by used for tracking use-cases.

Frequency of pings

- When rider is delivering order, pings will be published every 10s.

A. API Contract

Order Track API Header

```
HTTP Method : GET
Endpoint : <host_url>/v1/orders/<zomato_internal_order_id>/track
Header : {
  Content-Type : application/json
  Authorization : {{access_token}}
}
```

Order Track API Response

- If Order Id is not found in system

```
{
    "errors": "Order ID not found.",
    "status": {
        "code": 500
    }
}
```

- If Order is cancelled.

```
{
    "order_status": "CANCELLED",
    "status": {
        "code": 200
    },
    "external_order_id": "558647351",
    "tracking_link": "",
    "driver_name": "Neeraj Nayal",
    "driver_phone": "8335881012"
}
```

- Other Order Status.

A. Order is created but not assigned to Rider.

```
{
    "order_status": "CREATED",
    "status": {
        "code": 200
    },
    "external_order_id": "1436230654"
}
```

B. Order is Confirmed by Rider.

```
{
    "order_status": "CONFIRM",
    "status": {
        "code": 200
    },
    "location": {
        "latitude": 12.334308624267578,
        "longitude": 76.57892608642578
    },
    "external_order_id": "1931472461",
    "tracking_link": '',
    "driver_name": "Vinod Kumar G",
    "driver_phone": "9006564846",
    "Eta": {
        "pickup_eta": 10.89,
        "drop_eta": 13.53,
        "total_eta": 31.53,
        "drop_distance": 3,
        "pickup_distance": 10,
        "polyline": "gsahs_yrhda!jcd"
    }
}
```

C. Rider reached shop.

```
{
    "order_status": "REACHED_SHOP",
    "status": {
        "code": 200
    },
    "location": {
        "latitude": 28.1927490234375,
        "longitude": 76.81423950195312
    },
    "external_order_id": "2057718515",
    "tracking_link": "",
    "driver_name": "Surender Singh",
    "driver_phone": "000",
    "Eta": {
        "pickup_eta": 0.0,
        "drop_eta": 6.95,
        "total_eta": 9.95,
        "drop_distance": 3,
        "pickup_distance": 0,
        "polyline": "gsahs_yrhda!jcd"
    }
}
```

```

D. Rider is out for delivery.
{
    "order_status": "SHIPPED",
    "status": {
        "code": 200
    },
    "location": {
        "latitude": 29.467388153076172,
        "longitude": 77.6961441040039
    },
    "external_order_id": "1792105760",
    "tracking_link": "",
    "driver_name": "Shamshad",
    "driver_phone": "00000000",
    "Eta": {
        "pickup_eta": 0.0,
        "drop_eta": 13.81,
        "total_eta": 15.81,
        "drop_distance": 3,
        "pickup_distance": 0,
        "polyline": "gsahs_yrhda!jcd"
    }
}


E. Order is delivered and marked complete.
{
    "order_status": "COMPLETE",
    "status": {
        "code": 200
    },
    "external_order_id": "1951754555",
    "tracking_link": "",
    "driver_name": "Ravi Chandiran",
    "driver_phone": "00000"
}


order_status can be
CREATED
CONFIRM
REACHED_SHOP
SHIPPED
REACHED_DROP
COMPLETE
CANCELLED
RETURN
```

Advanced

OTP Handshake for Return OTP

```
To ensure OTP handshake in return order flow, Order detail should have below fields
"return_otp": "4 digit OTP"

```

```
Example :
"order_details":{
      "order_value":"150",
      "order_type":{
         "name":"Prepaid"
      },
      "category": {
         "name": "FOOD"
      },
      "order_id":"Test84fnf4039",
      "amount_to_be_collected":"0",
      "amount_to_be_paid":"0",
      "pickup_otp": "3746",
      "return_otp": "4321",
      "order_items":[
         {
            "item":{
               "name":"Chai"
            },
            "weight":100, // In grams
            "quantity":3,
            "price": 50
         },
         {
            "item":{
               "name":"Besan"
            },
            "weight":1000, // In grams
            "quantity":1,
            "price":100
         }
      ]
   }
```

UAT Guidelines

Login to staging dashboard using your email id and password (will be shared post merchant registration). Post login generate client key and secret from the dashboard.

All the below listed apis can only be used in staging environments, using these apis in production environment will result in `422` status code (`UNPROCESSABLE_ENTITY`)

Host URL: `https://staging.runnr.in`

Order Details API

Use this api to validate the payload sent in order creation.

Order Details API Request

```
HTTP Method: GET
Endpoint : <host_url>/test/order/order_details?order_id={order_id}
Header : {
  Content-Type : application/json
```

```
  Authorization : {{access_token}}
}
```

Order Details API Response


```
{
    "status": {
        "code": 200
    },
    "order_details": {
        "id": "f71f53be28139a86c2",
        "external_order_id": "b2b_uat_test",
        "created_at": "2023-12-29T14:18:04.000+05:30",
        "order_type": "Prepaid",
        "pickup_user": {
            "id": 156370490,
            "name": "User",
            "phone": "XXXXX0002",
            "email": "",
            "address_details": {
                "address": "Zomato office Koramangala",
                "latitude": "12.9352",
                "longitude": "77.6245"
            }
        },
        "drop_user": {
            "id": 156370491,
            "name": "User",
            "phone": "XXXXXX0011",
            "email": "",
            "address_details": {
                "address": "Nexus Mall",
                "latitude": "12.9344",
                "longitude": "77.6113"
            }
        },
        "callback_url": "test.in/order_callback",
        "order_items": [
            {
                "name": "Chai",
                "quantity": 1,
                "price": 10.0
            }
        ],
        "order_options": {
            "preparation_time": 5.0,
            "drop_user_instruction": ""
        },
        "otp_details": {
            "pickup_otp": "1234",
            "drop_otp": "",
            "return_otp": ""
        }
    }
}
```

Order Callback API

Use this api to move the order state from one to another. The state transition should follow the state machine. The state machine for different journeys are listed below :-

Order complete journey:

`TRIP_ASSIGNED -> DRIVER_ASSIGNED -> REACHED_PICKUP -> SHIPPED -> DRIVER_REACHED_DROP -> COMPLETE`

Order cancel journey:

`CANCELLED`

`TRIP_ASSIGNED -> CANCELLED`

`TRIP_ASSIGNED -> DRIVER_ASSIGNED -> CANCELLED`

`TRIP_ASSIGNED -> DRIVER_ASSIGNED -> REACHED_PICKUP -> CANCELLED`

Return order journey:

`TRIP_ASSIGNED -> DRIVER_ASSIGNED -> REACHED_PICKUP -> SHIPPED -> CANCELLED -> RETURN`

`TRIP_ASSIGNED -> DRIVER_ASSIGNED -> REACHED_PICKUP -> SHIPPED -> DRIVER_REACHED_DROP -> CANCELLED -> RETURN`

Return cancelled journey:

`TRIP_ASSIGNED -> DRIVER_ASSIGNED -> REACHED_PICKUP -> SHIPPED -> CANCELLED -> RETURN_CANCELLED`

`TRIP_ASSIGNED -> DRIVER_ASSIGNED -> REACHED_PICKUP -> SHIPPED -> DRIVER_REACHED_DROP -> CANCELLED -> RETURN_CANCELLED`

supported events: `[DRIVER_ASSIGN_FAILED, TRIP_ASSIGNED, DRIVER_ASSIGNED, REACHED_PICKUP, SHIPPED, DRIVER_REACHED_DROP, COMPLETE, CANCELLED, RETURN, RETURN_CANCELLED, RAIN]`

1. To trigger rain event callback use event as `“RAIN"`
2. To trigger `DRIVER_ASSIGN_FAILED` order status callback use event as `“DRIVER_ASSIGN_FAILED"`

Order Callback API Header

```
HTTP Method: POST
Endpoint: <host_url>/test/order/callback
Header : {
  Content-Type : application/json
  Authorization : {{access_token}}
}
```

Order Callback API Request Payload

```
{
  "order_id": "210f29da7e437417d0",
  "event": "DRIVER_ASSIGNED"
}
```

Order Callback API Response

Success:

```
{
    "status": {
        "code": 200
    }
}
```

Bad Request:

```
{
    "status": {
        "code": 400,
        "message": "order is already in terminal state"
    }
}
```

State machine violated:

```
{
    "status": {
        "code": 500
        "message": "state machine violated"
    }
}
```

Internal server error:

```
{
    "status": {
        "code": 500,
        "message": "something went wrong"
    }
}
```